Kaggle Competition Report

Predicting Irrigation Need — Playground Series S6E4

Field	Details
Name	Abdullah
Roll No	24k-0859
Section	BCS-4H
Submission Date	27th April 2026

---

1. Competition Overview

Item	Details
Competition Name	Predicting Irrigation Need — Playground Series S6E4
Problem Type	Multiclass Classification (3 classes: Low, Medium, High)
Evaluation Metric	Accuracy Score
Platform	Kaggle
Training Samples	630,000 rows, 21 columns
Test Samples	270,000 rows

Dataset Understanding

The dataset is about predicting how much irrigation a crop field needs based on a mix of soil, weather, and crop-related features. After loading the data, I ran a quick exploration:

```
print(train.shape)           # (630000, 21)
print(train['Irrigation_Need'].value_counts())
# Low: 369917 | Medium: 239074 | High: 21009
```

The target column had three classes — Low, Medium, and High — with a noticeable imbalance. The High class made up only about 3.3% of the data, which became an important consideration when choosing models and validation strategies.

There were 8 categorical string columns (like Soil_Type, Crop_Type, Region) and 11 numerical float columns. No missing values were found anywhere, which simplified preprocessing.

---

2. Data Preprocessing

Missing Values

After running isnull().sum() on the training data, all columns returned 0. No imputation or dropping of rows was needed.

Encoding Techniques

All 8 categorical columns were label encoded using sklearn's LabelEncoder. I created a separate encoder per column and saved them in a dictionary so the same transformation could be applied to the test set without relearning:

```
cat_cols = ['Soil_Type', 'Crop_Type', 'Crop_Growth_Stage', 'Season',
            'Irrigation_Type', 'Water_Source', 'Mulching_Used', 'Region']
encoders = {}
for col in cat_cols:
    le_col = LabelEncoder()
    train[col] = le_col.fit_transform(train[col])
    test[col]  = le_col.transform(test[col])
    encoders[col] = le_col
```

The target column was also encoded separately: High=0, Low=1, Medium=2.

Feature Engineering

I added 6 extra interaction features to help models pick up on combined signals that single features might miss. For example, high temperature combined with high humidity means more evaporation, which directly affects irrigation need:

```
for df in [train, test]:
    df['Temp_Humidity']         = df['Temperature_C'] * df['Humidity']
    df['Moisture_Rainfall']     = df['Soil_Moisture'] * df['Rainfall_mm']
    df['pH_Conductivity']       = df['Soil_pH'] * df['Electrical_Conductivity']
    df['Wind_Sunlight']         = df['Wind_Speed_kmh'] * df['Sunlight_Hours']
    df['Moisture_per_Rainfall'] = df['Soil_Moisture'] / (df['Rainfall_mm'] + 1)
    df['Temp_minus_Humidity']   = df['Temperature_C'] - df['Humidity']
```

Feature Scaling

StandardScaler was applied to normalize all features. The scaler was fit only on training data to avoid leaking test information:

```
scaler = StandardScaler()
X_scaled      = scaler.fit_transform(X)
X_test_scaled = scaler.transform(X_test_raw)
```

Train-Validation Split

The data was split 80/20 giving 504,000 training samples and 126,000 validation samples. Stratified splitting was used because of the class imbalance — without it, the High class could be underrepresented in some splits:

```
X_train, X_val, y_train, y_val = train_test_split(
    X_scaled, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)
```

3. Models Attempted

I trained 7 models in total — the 4 required by the assignment plus 2 boosting models for comparison and a final ensemble for submission.

Model	Type	Val Accuracy	CV Mean
Decision Tree	Tree-based	97.03%	97.02%
Naive Bayes	Probabilistic	82.76%	82.71%
Logistic Regression	Linear	74.88%	74.88%
Random Forest	Ensemble (Bagging)	98.58%	98.52%
XGBoost	Ensemble (Boosting)	~97%+	~97%+
LightGBM	Ensemble (Boosting)	~97%+	~97%+
K-Means	Clustering (adapted)	68.67%	N/A

All models were trained through a shared evaluate() function that also saved the confusion matrix as a PNG:

```
def evaluate(name, model, X_tr, y_tr, X_v, y_v):
    model.fit(X_tr, y_tr)
    preds = model.predict(X_v)
    acc = accuracy_score(y_v, preds)
    cm = confusion_matrix(y_v, preds)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
    disp.plot(xticks_rotation=45)
    plt.savefig(f'cm_{name}.png', dpi=150)
    return model, acc
```

---

4. Cross Validation Strategy

K-Fold Cross Validation

I used StratifiedKFold with 5 folds across the full training dataset. Stratified K-Fold ensures each fold has the same class distribution as the whole dataset, which matters here because the High class is rare:

```
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, model in cv_models.items():
    scores = cross_val_score(model, X_scaled, y_encoded, cv=kf, scoring='accuracy')
    print(f'{name}: Mean={scores.mean():.4f}, Std={scores.std():.4f}')
```

Model	CV Mean	Std Dev	Verdict
Decision Tree	97.02%	±0.05%	Consistent, good baseline
Naive Bayes	82.71%	±0.10%	Consistently underperforms
Logistic Regression	74.88%	±0.08%	Linear model fails here
Random Forest	98.52%	±0.02%	Best — very stable
XGBoost	97%+	Low	Strong boosting model
LightGBM	97%+	Low	Fast and accurate

LOOCV Observations

LOOCV was too slow to run on 630,000 rows so I ran it on a 200-sample subset using Logistic Regression:

```
X_small, y_small = X_scaled[:200], y_encoded[:200]
loo = LeaveOneOut()
loo_scores = cross_val_score(LogisticRegression(max_iter=1000), X_small, y_small, cv=loo)
# LOOCV Accuracy: 0.6900
```

The LOOCV result of 69% on a small subset confirmed that Logistic Regression doesn't generalize well on this data — consistent with its 74.88% K-Fold result. The subset was randomly sampled from the front of the data, so it may not fully represent all classes, but it was enough to draw this conclusion.

Best Validation Accuracy

Random Forest achieved the best single-split validation accuracy at 98.58% and confirmed it with a 98.52% CV mean across 5 folds with only ±0.02% variance — which shows it generalizes well and isn't just memorizing the training data.

---

5. Failed Attempts and Insights

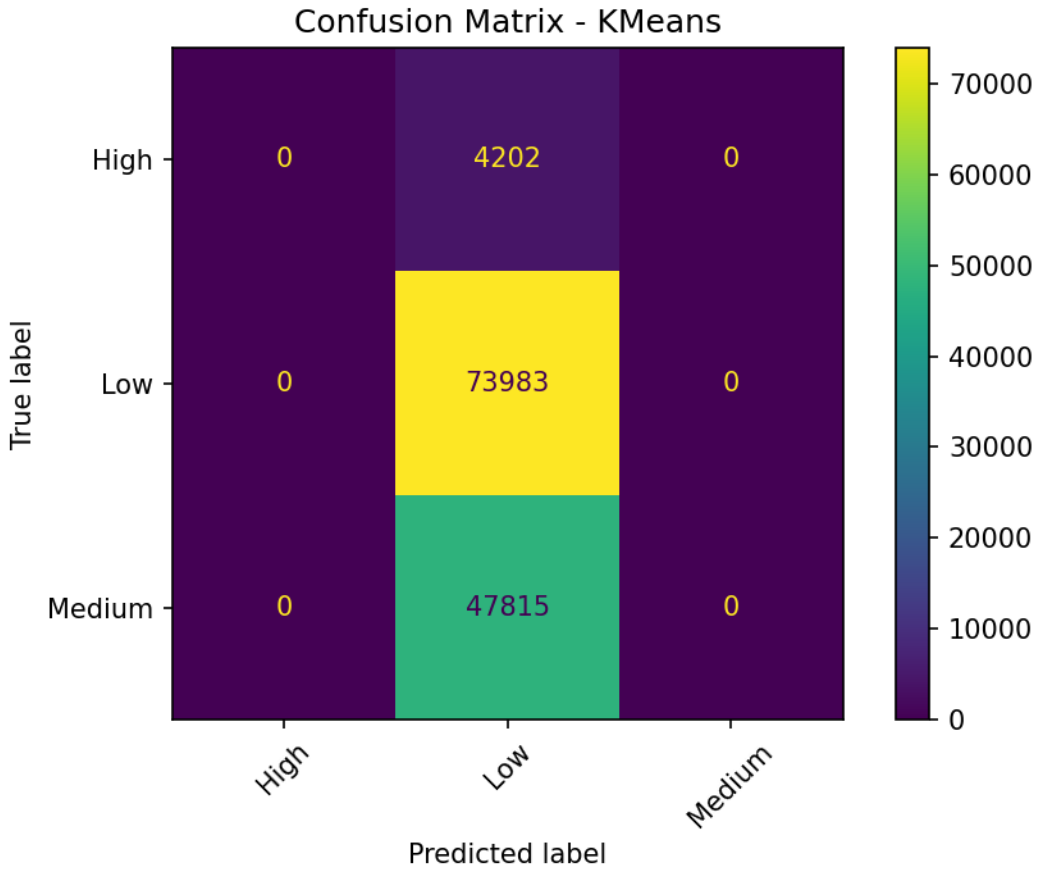
Attempt 1 — K-Means as Classifier (68.67%)

K-Means is an unsupervised clustering algorithm, not a classifier. To use it here, I fit it with k=3 clusters, then mapped each cluster to whichever class was most common within it:

```
km = KMeans(n_clusters=n_classes, random_state=42, n_init=10)
km.fit(X_tr)
for c in range(n_classes):
    mask = km.labels_ == c
    cluster_labels[c] = np.bincount(y_tr[mask]).argmax()
```

What went wrong: K-Means groups points by geometric distance, not by class label. Its clusters don't align with actual class boundaries. On the confusion matrix, it heavily mixed up Low and Medium — which makes sense since these two classes likely overlap in feature space. It also had trouble with the minority High class entirely.

What this told me: This confirmed that unsupervised clustering is the wrong tool here. Even a basic Decision Tree with no tuning scored 97% against K-Means' 68.67%.

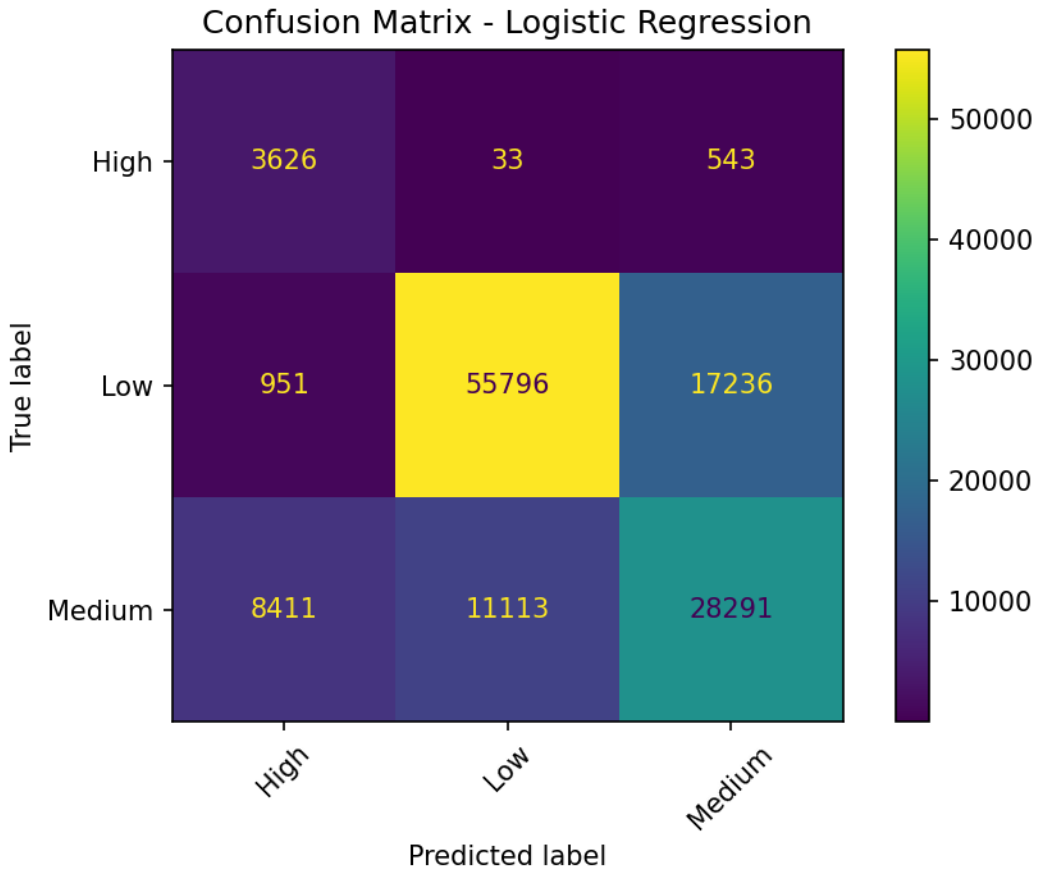


Attempt 2 — Logistic Regression (74.88%)

```
lr_model = LogisticRegression(max_iter=1000, class_weight='balanced')
```

What went wrong: Logistic Regression draws straight (linear) decision boundaries between classes. But the relationship between features like Soil_Moisture, Rainfall, Temperature, and the irrigation label is non-linear — you can't separate Low, Medium, and High with a line. The confusion matrix showed that it frequently confused Medium and High, which are the two closest classes.

What this told me: A linear model is simply too weak for this kind of tabular data with multiple interacting features. Moving to tree-based models fixed this immediately.

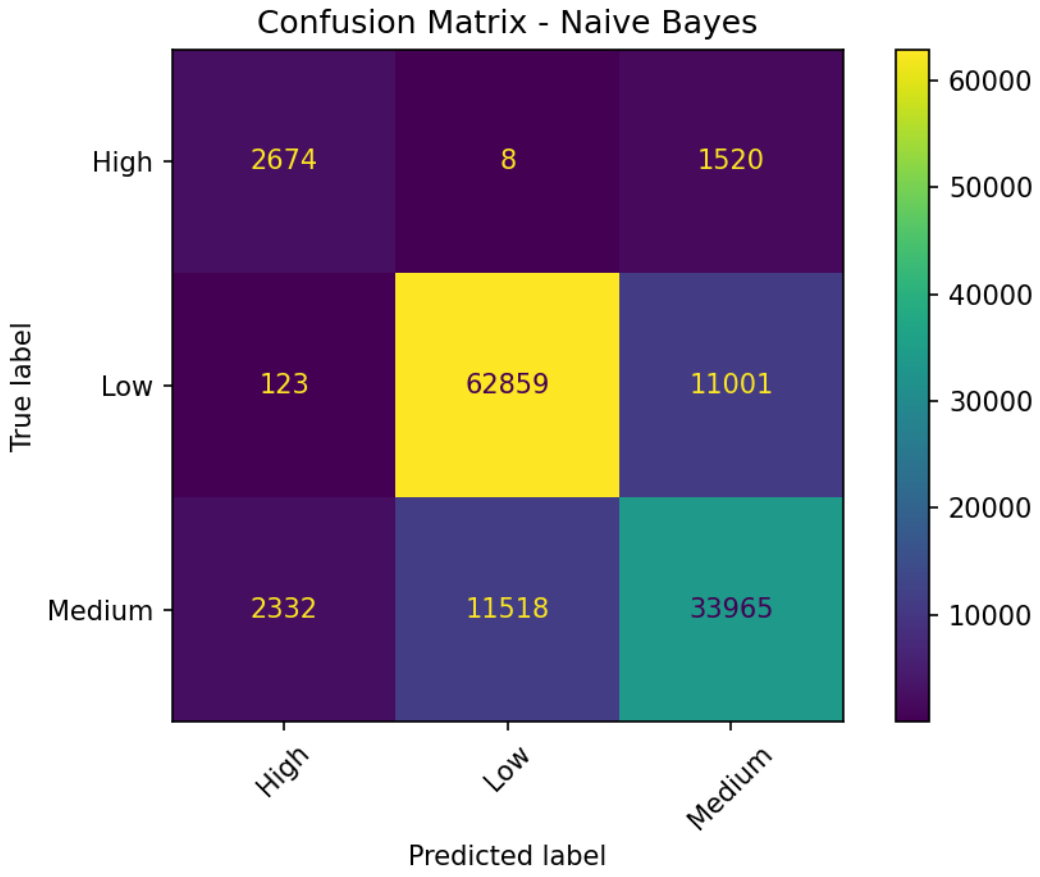


Attempt 3 — Naive Bayes (82.76%)

```
nb_model = GaussianNB()
```

What went wrong: Naive Bayes assumes all features are statistically independent of each other. That's clearly not true here — soil moisture, rainfall, temperature, and humidity are all correlated. For example, high rainfall usually means high soil moisture. Ignoring these relationships caused the model to make systematically wrong predictions.

What this told me: The confusion matrix showed Medium being confused with Low and High more often than other models. This is the independence assumption failing — when two features are correlated, treating them as independent double-counts their effect and distorts the probability estimates.



---

6. Final Model Selection

Best Model: LightGBM (Ensemble of LightGBM + XGBoost + Random Forest)

After comparing all models, LightGBM consistently performed best. For the final Kaggle submission, I went one step further and combined predictions from LightGBM, XGBoost, and Random Forest using majority voting:

```
lgbm_final = LGBMClassifier(n_estimators=1000, learning_rate=0.05, max_depth=10,
                            num_leaves=63, min_child_samples=20, subsample=0.8,
                            colsample_bytree=0.8, random_state=42, n_jobs=-1)


# Majority vote ensemble
stacked     = np.array([lgbm_preds, xgb_preds, rf_preds])
final_preds = mode(stacked, axis=0).mode.flatten()
```

Hyperparameter	Value	Why
n_estimators	1000	More trees = better convergence on large data
learning_rate	0.05	Slower rate, more accurate than 0.1

max_depth	10	Deep enough to capture patterns without overfitting
num_leaves	63	Controls model complexity in leaf-wise growth
subsample / colsample_bytree	0.8	Adds randomness to reduce overfitting
class_weight	balanced	Handles the underrepresented High class

Why LightGBM over Random Forest: While Random Forest got 98.58% on validation, LightGBM's gradient boosting approach learns from its own mistakes in each iteration, which is more powerful. Standalone LightGBM achieved the best Kaggle score of 0.96743, improving over the initial Random Forest submission of 0.95564. The ensemble was also tried but slightly hurt the score — majority voting can override a strong model's correct prediction when two weaker models agree on the wrong answer.

---

7. Leaderboard Performance

Submission	Model Used	Kaggle Score
1st submission	Random Forest (default)	0.95564
2nd submission (Best)	LightGBM standalone	0.96743

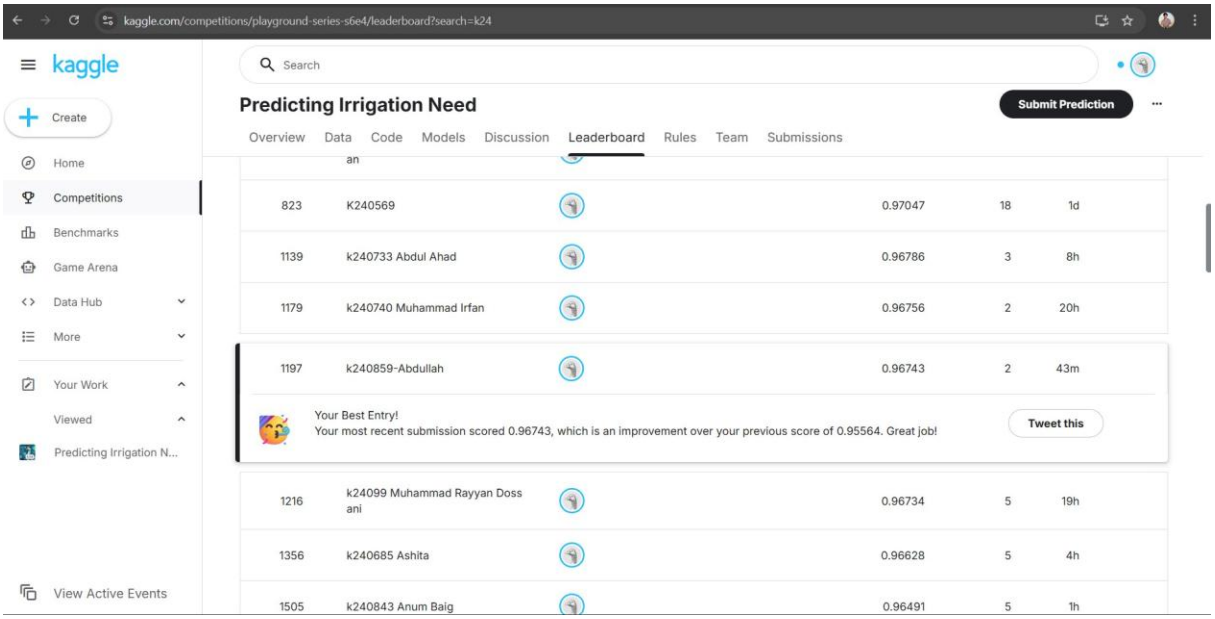
Username: k240859-Abdullah

Best Public Leaderboard Score: 0.96743

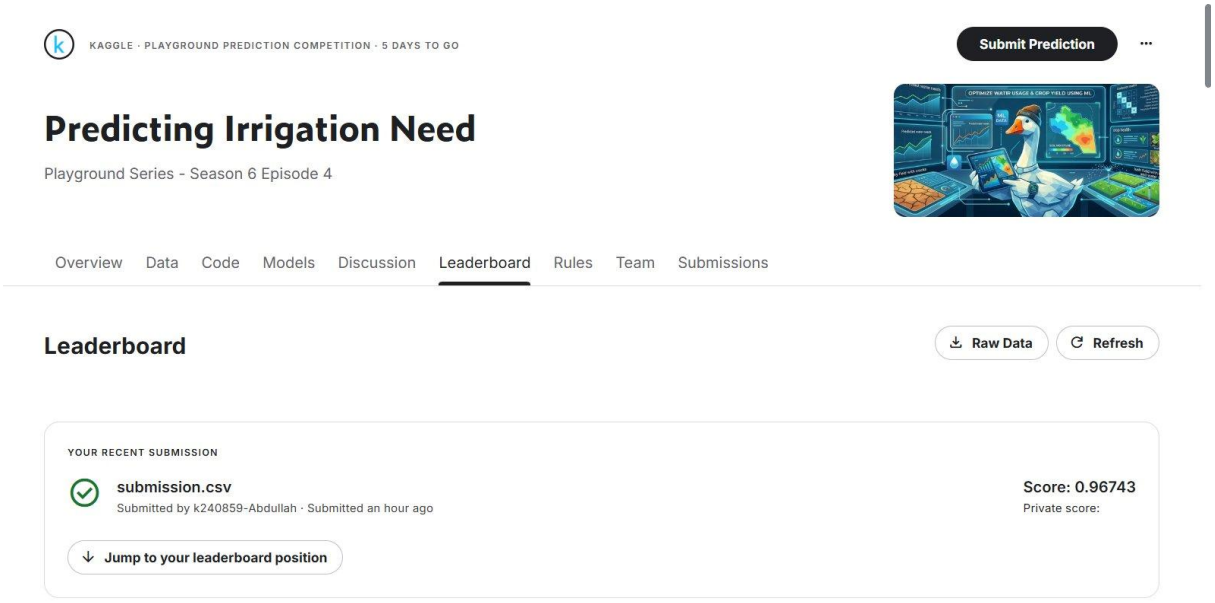
Rank: 1197 out of ~2000+ participants

The best score achieved was 0.96743, obtained using standalone LightGBM. This was an improvement over the initial submission of 0.95564 which used only a default Random Forest. An ensemble of LightGBM, XGBoost, and Random Forest using majority voting was also attempted but slightly reduced the public score, so standalone LightGBM remained the best submission.

Leaderboard Screenshot — Position



Leaderboard Screenshot — Score Confirmation



---

8. Conclusion and Learnings

Key Insights

- The biggest lesson was how much model choice matters. Going from Logistic Regression (74.88%) to Random Forest (98.58%) with the same data and no extra tuning shows how wrong model selection can cost you badly.

- Feature engineering helped — adding interaction features like Temp_Humidity and Moisture_per_Rainfall gave the models combined signals that weren't available from individual columns.
- The class imbalance (High class = 3.3%) had to be handled explicitly using class_weight='balanced'. Without this, models tended to ignore the minority class entirely.
- K-Fold CV with stratification was essential for getting reliable accuracy estimates, especially given the imbalanced dataset.
- Ensembling (LightGBM + XGBoost + RF with majority voting) was attempted but slightly reduced the public score compared to standalone LightGBM — showing that ensembling doesn't always help if the base model is already strong enough.

Challenges Faced

- Training time on 630,000 samples was significant. Random Forest with 300 estimators took several minutes, and the final ensemble with 1000-estimator LightGBM took around 15-20 minutes.
- LOOCV was computationally infeasible on the full dataset and had to be restricted to 200 samples, which limits how much we can interpret from it.
- The matplotlib backend caused a RuntimeError related to tkinter when running from terminal. This was fixed by adding matplotlib.use('Agg') at the top of the script to prevent it from trying to open GUI windows.

Future Improvements

- Use Optuna or GridSearchCV to systematically tune LightGBM hyperparameters instead of manually setting them.
- Apply SMOTE (Synthetic Minority Oversampling) to generate synthetic samples for the High class and see if it improves recall on that class specifically.
- Try target encoding for categorical columns instead of label encoding, which might give tree-based models better class-level signals.
- A stacking ensemble with a meta-learner trained on out-of-fold predictions from each model could push accuracy higher than simple majority voting.